DIM0501 — Boas Práticas de Programação

Relatório do Projeto da Unidade 1 — AgendaFácil

Prof. Marcus Terra · DIMAp/UFRN · Semestre 2026.1 · Entrega: 25/10/2026

Integrantes (nome e matrícula)	FRANCISCO EDSON DA COSTA FILHO (20240020730) JERBESON VINYCIUS SOUSA RODRIGUES (20230025296)
Linguagem	Python
Repositório ou arquivo entregue	`https://github.com/JerbesonViny/boas-praticas-projeto-faculdade`

1. Diagnóstico do código-base

#	Categoria	Anomalia e onde está	Por que é um problema
1	Programação defensiva	Em `add`, entradas inválidas são ignoradas silenciosamente sob um bloco `try/except: pass` genérico.	O chamador acredita que a consulta foi marcada quando na verdade falhou. Além disso, oculta bugs internos do próprio código.
2	Manutenibilidade	Parâmetros com nomes não descritivos em `add(p, pr, d, h, dur)`.	Exige leitura interna da função para deduzir os tipos e propósitos de cada variável.
3	Manutenibilidade	Acesso aos dados do agendamento por índices numéricos (ex.: `a[0]`, `a[1]`, `a[2]`).	Código frágil: alterar a estrutura ou a ordem da lista quebra a aplicação silenciosamente em múltiplos pontos.
4	Adequação e lógica	Ausência de validação para o formato de data (AAAA-MM-DD) nas funções de entrada.	Permite o cadastro de datas inválidas ou em formatos inconsistentes (ex.: `05/10/2026` ou `2026-02-30`).
5	Adequação e lógica	Duração aceita valores arbitrários (não valida o intervalo de 15 a 120 min nem múltiplos de 15).	Fere as regras de negócio da clínica, permitindo agendamentos com durações incoerentes (ex.: 1 min ou 500 min).
6	Desempenho e recursos	Varredura sequencial direta da lista global `ag`, duplicada em `add`, `rem` e `lst`.	Duplicação da lógica de consulta e ausência de uma camada isolada para a busca de registros.
7	Organização e estética	Acoplamento de responsabilidades em `add` e `rem` (validação, busca, mutação e `print`).	Viola o Princípio da Responsabilidade Única (SRP), dificultando o reuso e os testes isolados.
8	Programação defensiva	Estrutura de `if` profundamente aninhados dentro de `add`.	Aumenta a complexidade cognitiva e dificulta a manutenção em comparação ao uso de cláusulas de guarda.
9	Organização e estética	Código morto em comentário (`# def desmarcar`), TODO antigo e cabeçalho com histórico manual.	Gera poluição visual. O histórico de alterações pertence ao Git, e não ao cabeçalho do arquivo.
10	Organização e estética	Comentário impreciso em `lst` (`# lista os atendimentos do mes`).	Comentário incorreto que induz ao erro, pois a função filtra os atendimentos por um dia específico.

2. O que foi resolvido e o que ficou como dívida

#	Situação	Como foi resolvida, ou por que ficou
1	Resolvida	Removido o `try/except: pass`. Cada regra violada lança uma exceção específica (`HorarioForaIntervaloPermitidoErro` etc.).
2	Resolvida	Parâmetros renomeados para `nome_paciente`, `nome_doutor`, `data` etc., seguindo a PEP 8.
3	Resolvida	Estrutura convertida para a classe `Agendamento`, com atributos nomeados, e o status passou a usar `AgendamentoStatus`.
4	Resolvida	Criado `validar_data`, que exige estritamente AAAA-MM-DD por conversão de ida e volta.
5	Resolvida	Criado `validar_duracao`, que exige de 15 a 120 min, em múltiplos de 15 min.
6	Dívida assumida	Busca mantida por iteração. Como o volume de uma clínica é pequeno, o custo é irrisório. Fica centralizada em `__buscar_agendamento`.
7	Resolvida	Regras isoladas em `agenda_utilitario` e `data_utilitarios`, aplicadas pela classe `Agendamento`; controle de estado na classe `Agenda`, sem `print`.
8	Resolvida	Estrutura alterada para cláusulas de guarda (*guard clauses*), com exceções imediatas.
9	Resolvida	Removidos o TODO, o código comentado e o histórico manual do cabeçalho.
10	Resolvida	Substituído por docstring no padrão Google detalhando a busca por data.

3. Decisões de projeto

Decisão tomada	Alternativa descartada	Por que decidimos assim
Validação no construtor do `Agendamento` (`se_validar`).	Validar apenas na classe `Agenda`.	Evita a existência de instâncias inconsistentes no sistema.
Exceções personalizadas, uma por regra.	Lançar apenas `ValueError` genérico ou mensagens de texto.	Permite o tratamento granular dos erros pela camada chamadora.
Validação de data por ida e volta (`strptime` + `strftime`).	Expressão regular ou `date.fromisoformat`.	Rejeita formatos ambíguos (ex.: `2026-1-5`) sem a complexidade de uma regex.
Ao remarcar, o status passa a ser `REMARCADO`.	Manter o status `MARCADO`, como no código-base.	A mudança é proposital: o status deve refletir o estado real do agendamento. Manter `MARCADO` após uma remarcação esconderia essa informação de quem consulta a agenda.

4. Mudanças de comportamento

Entrada ou situação	O que acontecia antes	O que acontece agora
Horário fora do expediente (ex.: 20h)	Falhava em silêncio	Lança `HorarioForaIntervaloPermitidoErro`
Remarcar para horário inválido	Remarcava sem validar	Lança `HorarioForaIntervaloPermitidoErro`
Duração fora de 15 a 120 min	Aceitava silenciosamente	Lança `ConsultaIntervaloDuracaoErro`
Duração não múltipla de 15 min	Aceitava	Lança `ConsultaMultiploDuracaoErro`
Atendimento que termina depois das 18h (ex.: 17h com 120 min)	Aceitava	Lança `ConsultaTerminoForaExpedienteErro`
Horário ocupado ao marcar ou remarcar	Imprimia `Horario ocupado`	Lança `HorarioReservadoErro`
Remarcar agendamento inexistente	Saía em silêncio	Lança `AgendamentoNaoEncontradoErro`
Data em formato incorreto (ex.: `2026-1-5`)	Aceitava como texto solto	Lança `FormatoDataInvalidaErro`
Data inexistente (ex.: `2026-02-30`)	Aceitava	Lança `ValueError`
Data, hora ou duração com tipo errado (ex.: `None`, `8.5`)	Aceitava	Lança `TypeError`
Erro inesperado interno	Engolido por `except: pass`	Propaga a exceção para o chamador

5. Como executar e gerar a documentação

Item	Resposta
Comando para compilar e executar	`python3 -m src.agenda`
Comando para gerar a documentação de API	`pdoc src -o docs`
Onde a documentação gerada ficou no repositório	Diretório `docs/`
Saída do programa de demonstração	(ver abaixo)

Saída do programa de demonstração (preservada):

```
Atendimento marcado para Ana
Horario ocupado
Atendimento marcado para Carla
Remarcado
14 Carla Dra. Souza MARCADO
```

6. Uso de ferramentas de IA

Pergunta	**Resposta**
Foram usadas ferramentas de IA neste trabalho?	☐ Não ☒ Sim
Em quais etapas, e para quê?	Revisão da conformidade com a PEP 8, apoio na estrutura das docstrings no padrão Google e formatação do relatório.